Modelo



Setembro de 2024

Eduardo Lourenço da Silva, Gabriel Pereira Lugon Moulin, João Pedro Novak Rausis e Vitor Vieira Machado

Pipeline de dados de criptomoedas e mercado financeiro com Machine Learning

Introdução e Objetivos

Visão Geral dos Requisitos

O PucCrypto é uma aplicação distribuída que permite cadastrar usuários, manter um catálogo de criptmoedas, montar uma lista pessoal de acompanhamentos e consultar históricos acompanhados de previsões.

ID	REQ Funcional
RF01	Permitir cadastro e autenticação de usuários com emissão de JWT.
RF02	Manter o CRUD completo do catálogo de criptomoedas.
RF03	Permitir que cada usuário mantenha sua lista pessoal e anotações.
RF04	Coletar da CoinGecko o histórico e os preços atuais dos ativos cadastrados.
RF05	Manter CRUD e consulta cronológica dos pontos de preço no MarketData.
RF06	Gerar previsão de 1 a 30 períodos a partir de uma série histórica.
RF07	Agregar lista, histórico e previsão em uma única resposta para o dashboard.
RF08	Disponibilizar interface web em microfrontends para autenticação, catálogo e dashboard.

Objetivos de Qualidade

ID	Meta	Descrição
QG01	Confiabilidade	Uma falha temporária da API não pode corromper nem duplicar o histórico.
QG02	Rastreabilidade	Todo indicador deve ser rastreável até a cotação lote fonte e versão do modelo.
QG03	Atualização	Um lote válido deve aparecer no dashboard poucos minutos após a coleta.
QG04	Escalabilidade	Novos ativos e maiores volumes devem ser processados sem redesenho estrutural.
QG05	Segurança	Credenciais devem permanecer fora do código e acessos devem seguir menor privilégio.

Partes Interessadas

Parte interessada	Contato	Expectativa
Usuário final/analista	Canal de suporte da plataforma	Consultar dados históricos, indicadores e sinais de mercado de forma clara e confiável.
Equipe de desenvolvimento	Canal interno da equipe técnica	Evoluir e corrigir o sistema com baixo impacto e boa manutenibilidade.
Administrador	Canal administrativo	Gerenciar configurações, integraçõesefuncionamento geral do sistema.
Provedor de dados de mercado	Suporte / documentação da API	Disponibilizar dados consistentes, atualizados e em

		formato adequado para integração.

Restrições Arquiteturais

Categoria	Restrição
Dados	Database per Service: PostgreSQL no Identity, Azure SQL no Catalog e MongoDB no MarketData..
Integracão	O frontend chama somente o BFF por meio de um API Gateway gerenciado.
Arquitetura	Clean Architecture combinada com Vertical Slice e dependências apontando para dentro.
Mensageria	RabbitMQ com mensagens JSON e consistência eventual entre Catalog e MarketData.

Contexto e Escopo

Contexto Negocial

O usuário interage com o PucCrypto pelo navegador. A plataforma administra identidade, catálogo, lista pessoal, histórico de preços e previsões. A CoinGecko permanece externa e fornece dados de mercado. Serviços gerenciados de nuvem hospedam o gateway, os bancos e o broker, mas não participam das regras de negócio.



Figura 01: Diagrama C4 nível 1

Elemento	Entrada para a plataforma	Saída da plataforma
Usuário final	Consultas, filtros por ativo, período e indicador	Sessão, catálogo, histórico, gráfico e previsão.
CoinGecko	Preços históricos e atuais em JSON	Requisições HTTP para ativos acompanhados
Serviço de nuvem	Conectividade, persistência e entrega de mensagens	Logs, conexões e consumo de recursos

Contexto Técnico

A interface web utiliza HTTPS e JSON. O API Gateway encaminha requisiçoes ao BFF. O BFF valida o JWT, repassa o token aos microsserviços e usa uma chave própria ao chamar a function. Catalog e MarketData se comunicam de forma assíncrona pelo RabbitMQ.

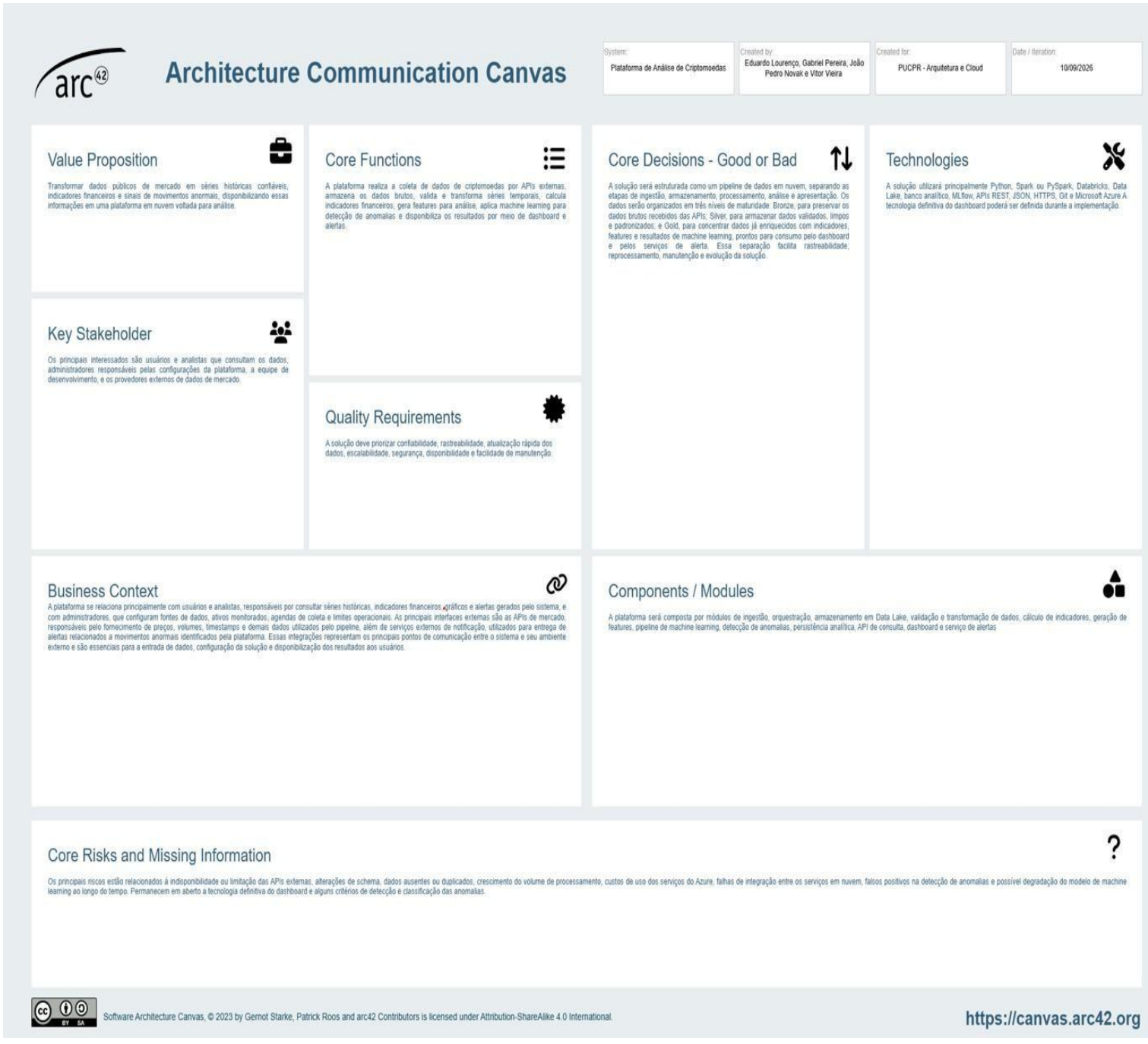
Entrada/Saída	Canal técnico	Protocolo + Formato
Navegador ↔ gateway/BFF	API web	HTTPS + JSON + JWT
BFF ↔ APIs internas	API REST	HTTP(S) + JSON + JWT
BFF ↔ Forecast	Azure Function HTTP	HTTPS+JSON+ Function Key -
Catalog ↔ MarketData	Broker de eventos	AMQP(S) + JSON
MarketData ↔ CoinGecko	API externa	HTTPS+JSON

Estratégia de Solução

A solução usa serviços pequenos e autonomos, cada qual proprietário de seus dados. Chamadas síncronas atendem as interações do usuário. Eventos mantem as réplicas necessárias entre Catalog e MarketData. O BFF adapta as APIs ao frontend e agrega o dashboard. Clean Architecture protege as regras internas, enquanto Vertical Slice aproxima os artefatos de cada uso.

Estratégia	Resumo
Decomposição	Identity, Catalog, MarketData, Forecast Function, BFF e quatro aplicações frontend.
Persistência	Um banco por serviço, com tecnologia escolhida de acordo com a responsabilidade.
Integração síncrona	REST/JSON por interfaces definidas na camada Application.
Integração assíncrona	Eventos CryptoRegistered, CryptoRemoved e PricesIngested pelo RabbitMQ.
Frontend	Shell e três remotos React carregados em tempo de execução por Module Federation.

Machine Learning	Regressão SDCA do ML.NET treinada sob demanda sobre variações históricas.
Qualidade	Testes unitários, de componentes e de arquitetura em cada componente.



Visão de Blocos de Construção

A plataforma é decomposta em blocos responsáveis pela coleta, armazenamento, processamento, análise e apresentação dos dados. A decomposição busca separar responsabilidades e facilitar escalabilidade, manutenção e evolução da solução.

Visão Sistêmica Geral de Caixa Branca

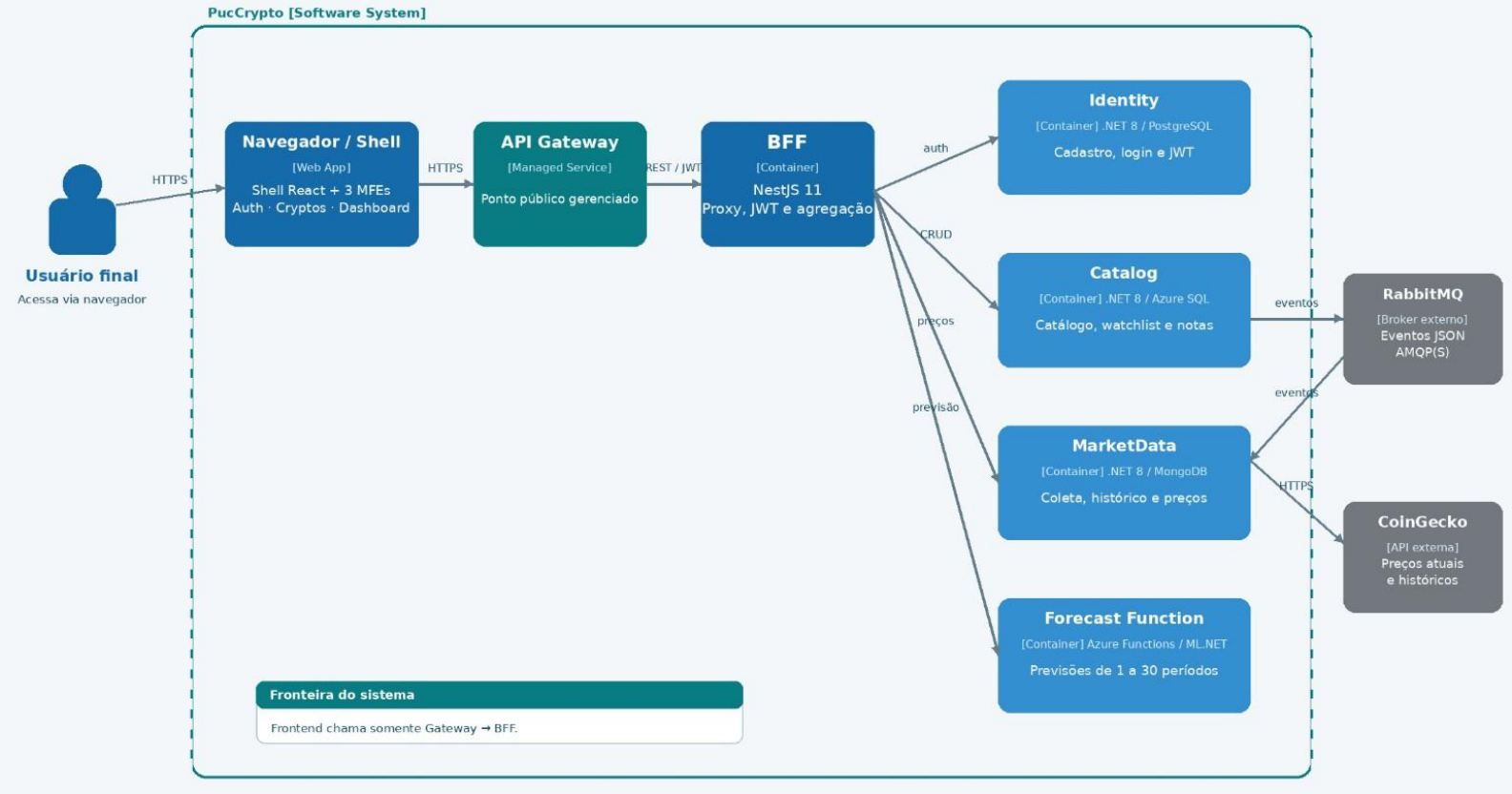


Figura 02: C4 nível 2- Contêiner

Motivação
A separação concentra autenticação, catálogo, mercado e previsão em unidades implantáveis independentes. O BFF é a única API conhecida pelo frontend, reduzindo seu acoplamento à topologia interna.

Nome	Responsabilidade
Shell	Navegação, sessão, tema, proteção de rotas e carga dos remotos.
MFE Auth	Cadastro e login.
MFE Cryptos	Catálogo, watchlist e anotações.
MFE Dashboard	Resumo, filtros, gráfico e tabela de histórico/previsão.
BFF	Proxy, autenticação de borda e agregação específica da interface.
Identity	Usuários, hash de senha e emissão de JWT.
Catalog	Criptomoedas, lista do usuário e último preço conhecido.
MarketData	Ativos acompanhados, coleta CoinGecko e pontos de preço.
Forecast Function	Previsão e disparo agendado da coleta.
RabbitMQ	Distribuição de eventos entre serviços.
Orquestrador	Agendar as coletas e processamentos, controlar retentativas e acompanhar as execuções

Nível 2 – Componentes do Catalog

O diagrama apresenta os componentes internos do microsserviço Catalog, organizados conforme Clean Architecture e Vertica Slice. Demonstra a comunicação com o BFF, a persistencia no Azure SQL e a integração assíncrona via RabbitMQ, promovendo baixo acoplamento e separação de responsabilidades.

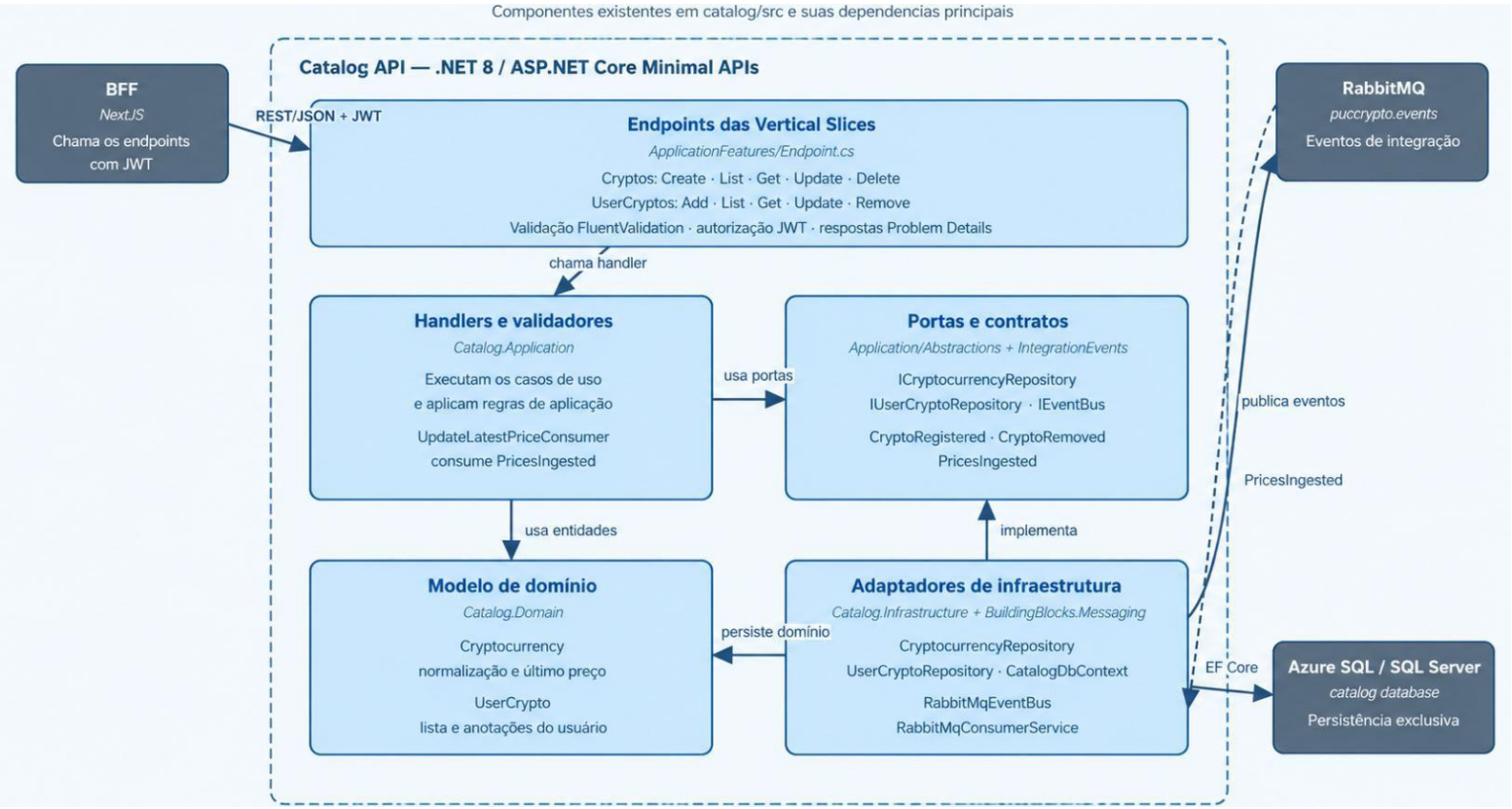


Figura 03: Diagrama C4 nível 3- Componentes do Catalog

Nome	Responsabilidade
BFF	Chamar os endpoints do Catalog por REST/JSON, encaminhando o JWT do usuário.

Endpoints das Vertical Slices	Expor os casos de uso de criptomoedas e da lista do usuário, aplicar validação, exigir autorização e converter resultados em respostas HTTP.
Handlers e validadores	Validar comandos e consultas, executar os casos de uso e coordenar as regras de aplicação do Catalog.
Portas e contratos	Definir as abstrações de persistência e mensageria, além dos contratos dos eventos de integração.
Modelo de domínio	Representar Cryptocurrency e UserCrypto, aplicando regras de normalização, atualização do último preço, lista pessoal e anotações.
Adaptadores de infraestrutura	Implementar persistência com Entity Framework Core e comunicação assíncrona com RabbitMQ.
RabbitMQ	Entregar os eventos CryptoRegistered, CryptoRemoved e PricesIngested entre o Catalog e outros serviços.
Azure SQL / SQL Server	Persistir exclusivamente os dados do Catalog, incluindo criptomoedas e itens das listas dos usuários.

Nível 3 – Implementação do Catalog

O nível de implementação detalha a estrutura interna da slice CreateCrypto, pertecente ao microsserviço Catalog. Apresenta os principais módulos, classes e dependências responsáveis pela leitura, validação, cadastro e persistencia das cryptomoedas, além da publicação dos eventos via integração RabbitMQ.

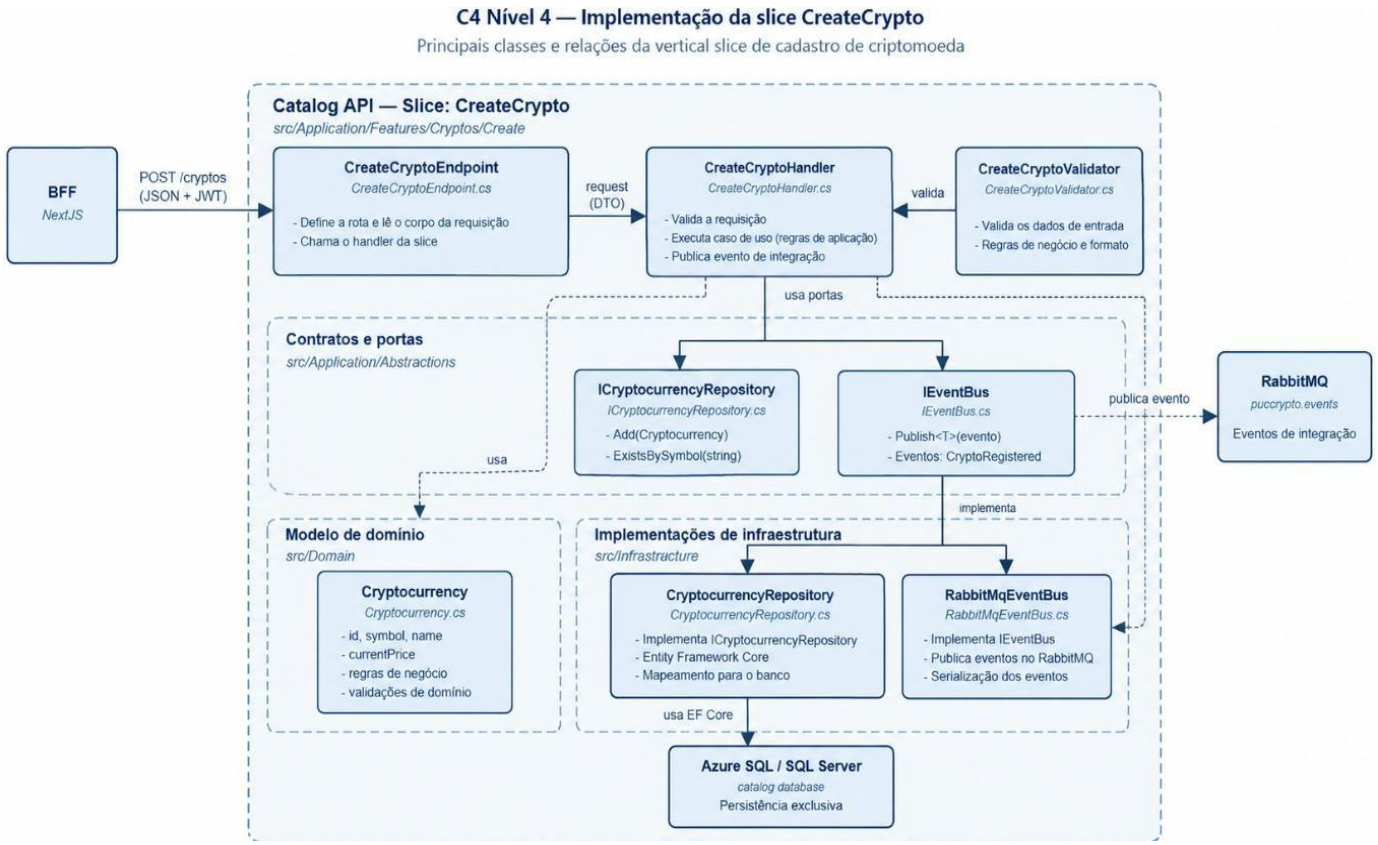


Figura 04: Diagrama C4 nível 4- Implementação da slice CreateCrypto.

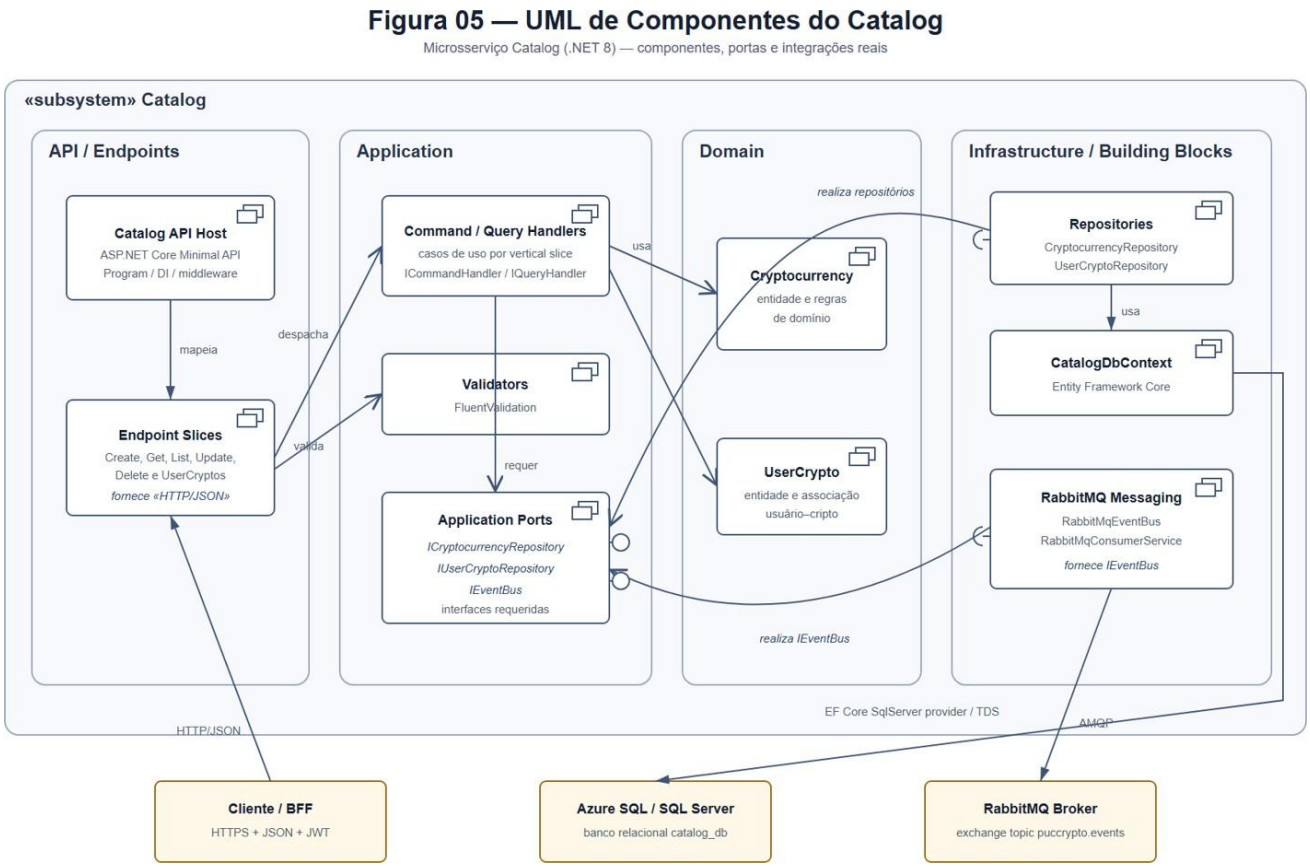


Figura 05: UML de componentes

Nome	Responsabilidade
Catalog API Host	Inicializar o microsserviço, configurar injeção de dependências, autenticação, middleware, Swagger, health checks, migrações e exposição da API HTTP.
Endpoint Slices	Mapear os endpoints HTTP das funcionalidades de criptomoedas e listas dos usuários, validar requisições e encaminhá-las aos handlers correspondentes.
Command / Query Handlers	Executar os casos de uso do Catalog, coordenando regras de domínio, persistência e publicação de eventos.
Validators	Validar comandos e requisições por meio do FluentValidation antes da execução dos casos de uso.
Application Ports	Definir as interfaces requeridas pela camada Application, como ICryptocurrencyRepository, IUserCryptoRepository e IEventBus.

Cryptocurrency	Representar a criptomoeda do catálogo e encapsular suas regras de criação, normalização, atualização e controle do preço mais recente.
UserCrypto	Representar a associação entre um usuário e uma criptomoeda monitorada, incluindo informações específicas dessa associação.
Repositories	Implementar as portas de persistência da Application para consultar e modificar criptomoedas e associações de usuários.
CatalogDbContext	Gerenciar o acesso aos dados do Catalog com Entity Framework Core, incluindo mapeamentos, rastreamento e persistência das entidades.
RabbitMQ Messaging	Implementar a publicação e o consumo de eventos de integração por meio do RabbitMQ, incluindo a realização da interface IEventBus.

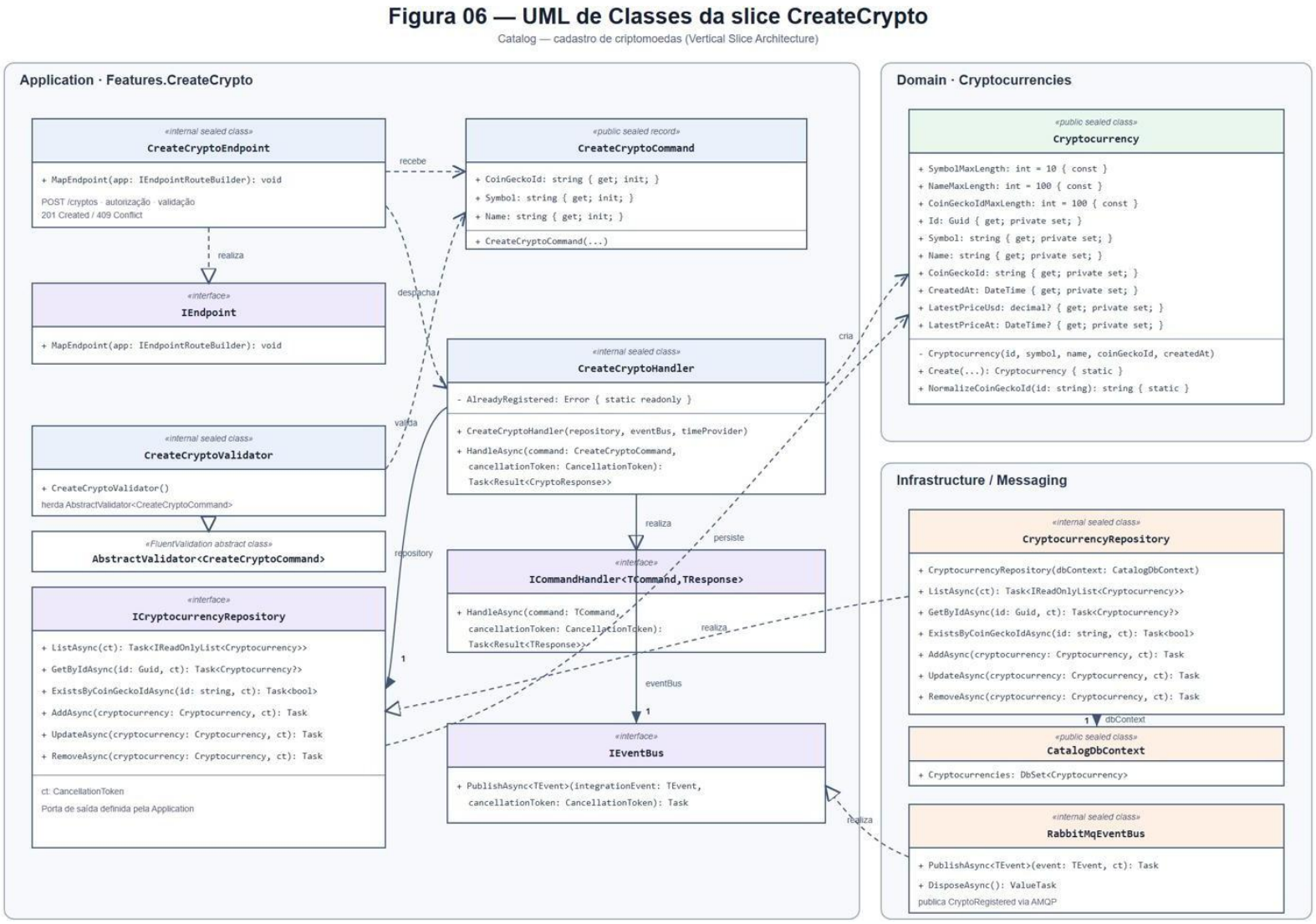


Figura 06: UML de classe

O diagrama de classe detalha as principais abstrações envolvidas na slice CreateCrypto, incluindo o CreateCryptoEndpoint como entrada HTTP, o CreateCryptoHandler, como coordenador do caso de uso e classes especializadas para validação, representação do comando, aplicação das regras de domínio, persistência e publicação de eventos. Essa estrututa busca manter responsabilidades bem definidas, reduzir o acoplamento por meio de interfaces e preservar a separação entre application, domain e infrastructure.

Visão de Tempo de Execução

A visão de tempo de execução descreve a interação entre os componentes do PucCrypto durante a consulta ao dashboard. O cenário principal apresenta a comunicação entre o frontend, BFF, Catalog, MarketData e forecast, desde a solicitação do usuário até a obtenção da lista de criptomoedas, histórico de preços e previsões

Cenário de consulta agregada ao dashboard

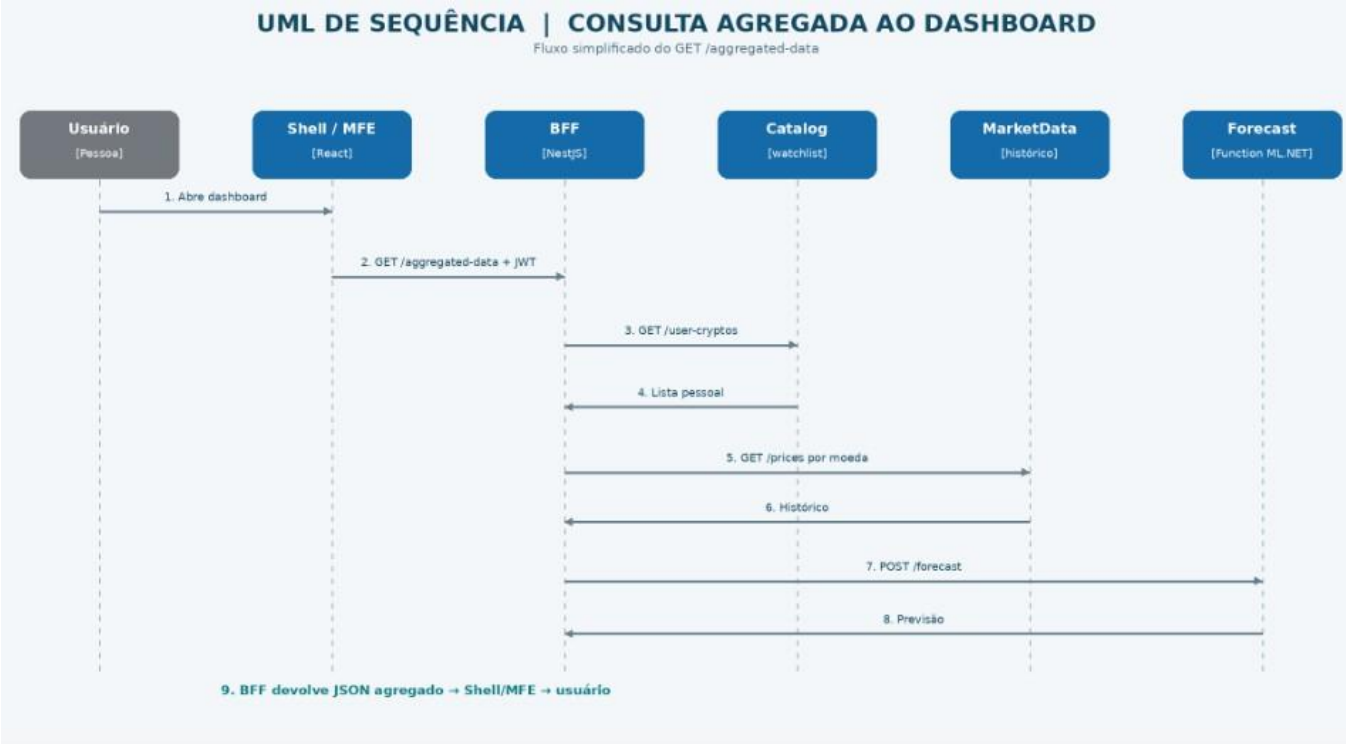


Figura 07: UML de sequência

Após validar o JWT, o BFF consulta a lista pessoal. Para cada moeda, busca o histórico no MarketData e quando há ao menos 14 observações distintas, envia a série a Function. As moedas são agregadas em paralelo. A indisponibilidade do MarketData ou da Function é convertida em estado parcial da moeda. Uma falha no Catalog interrompe a requisição porque sem a lista não há conteudo a agregar

Visão de Implantação

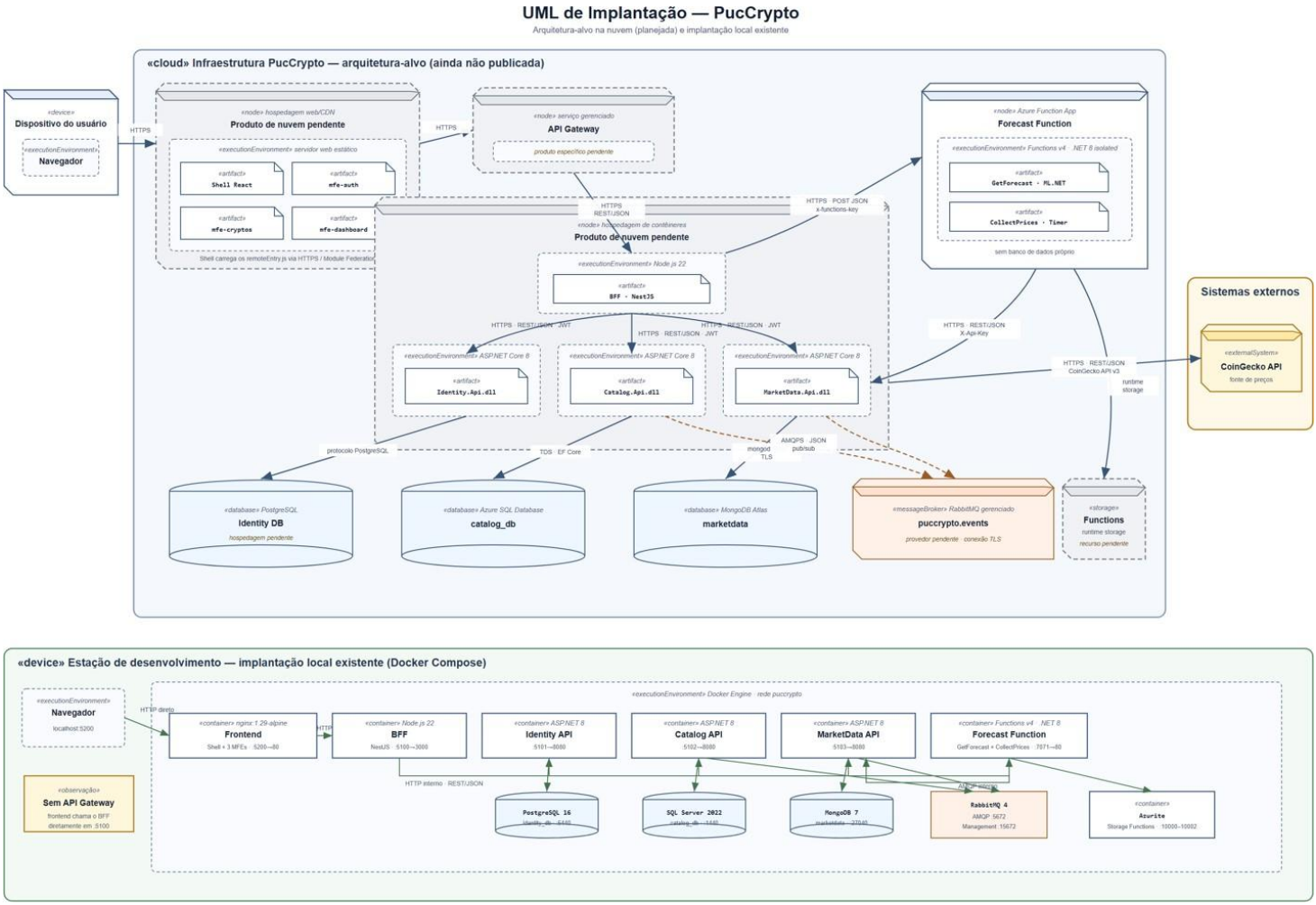


Figura 07: UML de implantação

O diagrama apresenta a implantação planejada do PucCrypto na nuvem e sua execução local. O frontend composto pelo Shell React e três microfrontends, acessa o BFF por meio de um API Gateway gerenciado. O BFF integra os microsserviços Identity, Catalog e MarketData e a Azure Function de previsão. Cada serviço possui sua própria tecnologia de persistências: PostgreSQL, Azure SQL e MongoDB Atlas. Catalog e MarketData comunicam-se de forma assíncrona pelo RabbitMQ, enquanto o MarketData consulta a API CoinGecko. Localmente, todos os elementos são executados com Docker Cimpose e o frontend acessa diretamente o BFF, sem API Gateway.

Conceitos Transversais

Modelo de dados

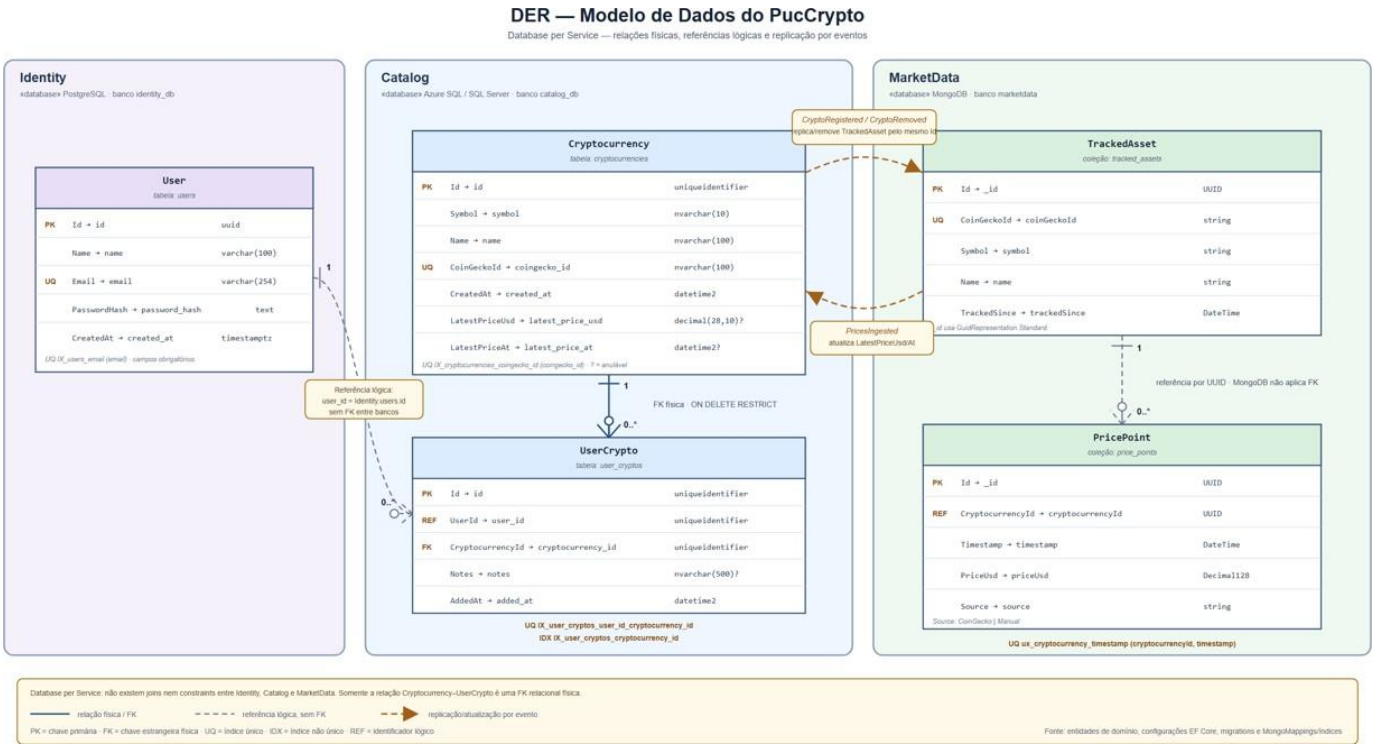


Figura 08: Diagrama de Entidades e Relacionamentos- DER

Organização e Persistência dos Dados

Identity e Catalog usam Entity Framework Core e migrations aplicadas na inicialização. MarketData usa o driver oficial do MongoDB e cria seus índices na inicialização. Forecast e BFF não possuem banco. Datas são tratadas em UTC e valores monetários são persistidos como decimal/Decimal128

Tratamento de Falhas

Os casos de uso retornam Result e erros são mapeados para Problem Details. Consumidores RabbitMQ rejeitam mensagens inválidas e realizam uma reentrega em falhas transitórias. O BFF preserva resultados parciais por moeda. O shell usa RemoteBoundary para impedir que a falha de um microfrontend derrube toda a aplicação

Autenticação e Autorização

O identity armazena senhas com BCrypt e emite JWT HS256. BFF, Catalog e MarketData validam emissr, audiência, assinatura e expiração. O BFF repassa o token ao chamar APIs protegidas. A coleta usa X-Api-Key e a previsão usa Function Key. A sessão do navegador permanece no sessionStorage e é encerrada ao receber 401

permitir identificar a versão utilizada, garantindo rastreabilidade e possibilitando comparação entre diferentes versões ao longo da evolução da solução

Configuração e Credenciais

Configurações são fornecidas por appsettings, variavéis de ambiente e arquivos de exemplo sem segredos reais. Connection strings, assinatura JWT, chaves da CoinGecko, coleta e Function devem vir do ambiente de implantação. Os valores de desenvolvimento do Compose são destinados apenas ao uso local.

Qualidade e Testes

Os serviços .NET possuem testes unitários e NetArchTest. O BFF usa Jest e regras de arquiteturas. O frontend possui Vitest, Testing Library e dependency-cruiser. As regras proíbem Domain de depender de frameworks, Application de depender de Infrastructure, referências entre slices e importações diretas entre microfrontends.

Decisões Arquiteturais

ADR	Decisão	Justificativa	Consequência
ADR01	Clean Architecture + Vertical Slice	Localizar cada caso de uso e impor dependências internas.	Application conhece abstrações HTTP mínimas; testes verificam fronteiras.
ADR02	Handlers próprios em vez de MediatR	Evitar dependência/licença e manter fluxo explícito.	Comportamentos transversais exigem decoradores próprios.
ADR03	Repositórios separados com workspace integrador	Atender à entrega e preservar execução conjunta.	Building Blocks são copiados entre repositórios.
ADR04	Um tipo de banco por serviço	Adequar persistência ao domínio e ao enunciado.	Operação usa três tecnologias de banco.
ADR05	Gateway gerenciado somente diante do BFF	Oferecer um único ponto de entrada ao frontend.	Todo acesso passa pelo BFF.

ADR06	Previsão em Function HTTP sem banco	Calcular sob demanda e simplificar propriedade dos dados.	Treinamento ocorre em cada chamada.
ADR07	RabbitMQ local e gerenciado	Usar o mesmo protocolo nos ambientes.	Requer operação do broker e tolerância à consistência eventual.
ADR08	Catálogo separado da lista do usuário	Representar corretamente os dois recursos.	Mais endpoints, com regras próprias.
ADR09	BFF NestJS com domínio e aplicação sem framework	Atender ao requisito Node mantendo arquitetura.	Há mapeamento entre DTOs e modelos internos.

Requisitos de qualidade

Árvore de qualidade

Categoria (ISO 25010)	Subcaracterística	Prioridade	Objetivos de qualidade relacionado
Confiabilidade	Tolerância a falhas e consistência dos dados	Alta	QG01
Segurança / Auditabilidade	Rastreabilidade dos resultados	Alta	QG02
Eficiência de desempenho	Tempo de atualização dos dados	Alta	QG03

Segurança	Confidencialidade e controle de acesso	Alta	QG05
Escalabilidade	Crescimento do volume e inclusão de novos ativos	Alta	QG04

Cenários de Qualidade

ID	Atributo de qualidade	Cenário	Resposta esperada	Métrica
CQ01	Confiabilidade	A Function falha para uma criptomoeda.	O dashboard entrega histórico e marca só a previsão como indisponível.	Demais moedas e dados disponíveis permanecem na resposta.
CQ02	Modificabilidade	Adicionar um novo caso de uso a um serviço.	Criar uma nova slice sem alterar slices existentes.	Nenhuma dependência entre slices detectada pelos testes.
CQ03	Usabilidade	Histórico insuficiente ou serviço indisponível.	A interface mostra estado específico e mantém navegação utilizável.	Sem tela em branco ou erro não tratado.
CQ04	Escalabilidade	Novos ativos ou maior volume de dados são adicionados	O sistema deve processar o aumento sem redesenho estrutural	Inclusão de novos ativos sem alteração da arquitetura; desempenho dentro do limite definido
CQ05	Segurança	Requisição sem token tenta acessar dados.	BFF/API rejeita a operação.	Resposta 401; nenhum dado protegido retornado.

Riscos e Débitos Técnicos

ID	Risco/Dívida técnica	Impacto	Mitigação/Plano
R01	Dependência de APIs externas de mercado	Indisponibilidade, limite de requisições ou mudança de schema pode interromper a coleta	Implementar tratamento de falhas, retentativas, validação de schema e desacoplamento entre coleta e processamento
R02	Modelo treinado a cada requisição.	Latência e consumo crescem com uso simultâneo.	Cache curto no BFF ou persistência/versionamento de modelos se necessário.
R03	JWT simétrico compartilhado.	Rotação exige coordenação entre serviços.	Migrar para assinatura assimétrica e descoberta de chaves em cenário real.
R04	Ausência de tracing e métricas centralizadas.	Diagnóstico de falhas distribuídas é mais lento.	Adotar OpenTelemetry, correlação e painéis operacionais.
R05	URLs de BFF e remotos definidas no build.	Mudança de ambiente exige rebuild do frontend.	Introduzir configuração em runtime.
R06	Implantação em nuvem ainda pendente.	A arquitetura alvo não está validada ponta a ponta publicamente.	Executar fase de deploy, smoke tests e atualização das URLs.
R07	Sem testes E2E versionados.	Regressões no fluxo completo podem escapar.	Adicionar Playwright ao CI com ambiente efêmero.
R08	Previsão financeira simplificada.	Usuários podem interpretar o resultado como recomendação.	Manter aviso explícito e não expor ações de negociação.